Name: Y.Veerendar Reddy **Reg.No:-** 192325102

Subject Code/Name: MLA0305-Reinforcement Learning

EXPERIMENT – 10

Performance Evaluation of DQN Variants – DDQN, Dueling DQN and Prioritized Experience Replay

Aim

To implement and compare different Deep Q-Network (DQN) variants, namely Double DQN (DDQN), Dueling DQN, and Prioritized Experience Replay, based on their learning performance.

Procedure

1. Create the CartPole Reinforcement Learning environment.
2. Build a standard DQN neural network.
3. Implement the DDQN method to reduce Q-value overestimation.
4. Implement the Dueling DQN architecture to separately estimate state value and action advantage.
5. Implement a simple Prioritized Experience Replay mechanism.
6. Train the agents for multiple episodes.
7. Record the reward obtained by each method.
8. Calculate the average performance.
9. Compare the methods using a graph.

Code:

```
# ============================================================
# EXPERIMENT 10
# DQN VARIANTS: DDQN, DUELING DQN & PRIORITIZED REPLAY
# ============================================================

import numpy as np
import tensorflow as tf
import gymnasium as gym
import matplotlib.pyplot as plt
from collections import deque
```

```
import random

np.random.seed(42)
tf.random.set_seed(42)

# ---------------------------------------------------------------
# Environment
# ---------------------------------------------------------------

env = gym.make("CartPole-v1")

state_size = env.observation_space.shape[0]
action_size = int(env.action_space.n)

gamma = 0.95
alpha = 0.001
epsilon_start = 1.0
epsilon_min = 0.05
epsilon_decay = 0.98
episodes = 30
batch_size = 32

# ---------------------------------------------------------------
# Models
# ---------------------------------------------------------------

def dqn_model():
    model = tf.keras.Sequential([
        tf.keras.layers.Input(shape=(state_size,)),
        tf.keras.layers.Dense(64, activation="relu"),
        tf.keras.layers.Dense(64, activation="relu"),
        tf.keras.layers.Dense(action_size)
    ])
    model.compile(optimizer=tf.keras.optimizers.Adam(alpha), loss="mse")
    return model
```

```
def dueling_model():
    inputs = tf.keras.Input(shape=(state_size,))
    x = tf.keras.layers.Dense(64, activation="relu")(inputs)
    x = tf.keras.layers.Dense(64, activation="relu")(x)

    value = tf.keras.layers.Dense(1)(x)
    advantage = tf.keras.layers.Dense(action_size)(x)

    advantage_mean = tf.keras.layers.Lambda(
        lambda a: a - tf.reduce_mean(a, axis=1, keepdims=True)
    )(advantage)

    output = tf.keras.layers.Add()([value, advantage_mean])
    model = tf.keras.Model(inputs, output)
    model.compile(optimizer=tf.keras.optimizers.Adam(alpha), loss="mse")
    return model

# ------------------------------------------------------------
# Training Function
# ------------------------------------------------------------

def train_agent(model, ddqn=False, prioritized=False):
    target_model = tf.keras.models.clone_model(model)
    target_model.set_weights(model.get_weights())

    memory = deque(maxlen=2000)
    rewards_history = []
    epsilon = epsilon_start

    for episode in range(episodes):
        state, _ = env.reset()
        total_reward = 0

        for step in range(500):
```

```
state_input = np.array([state])

# Epsilon-greedy
if np.random.random() < epsilon:
    action = env.action_space.sample()
else:
    action = np.argmax(model.predict(state_input, verbose=0)[0])

next_state, reward, terminated, truncated, _ = env.step(action)
done = terminated or truncated

memory.append((state, action, reward, next_state, done))
state = next_state
total_reward += reward

# Experience Replay
if len(memory) >= batch_size:
    if prioritized:
        batch = random.choices(
            list(memory),
            weights=[abs(x[2]) + 1 for x in memory],
            k=batch_size
        )
    else:
        batch = random.sample(list(memory), batch_size)

    states = np.array([x[0] for x in batch])
    next_states = np.array([x[3] for x in batch])

    targets = model.predict(states, verbose=0)
    next_q_online = model.predict(next_states, verbose=0)
    next_q_target = target_model.predict(next_states, verbose=0)

    for i, (_, a, r, _, d) in enumerate(batch):
        if d:
```

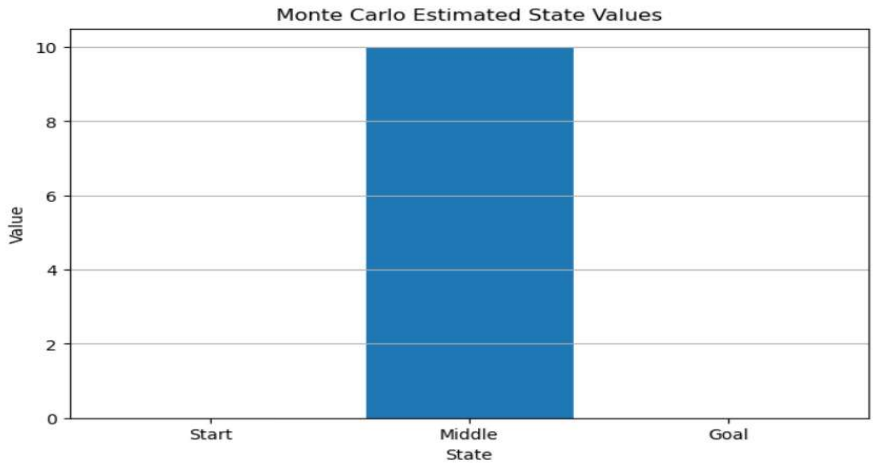
```
                targets[i, a] = r
            else:
                if ddqn:
                    # Double DQN: action from online, value from target
                    best_action = np.argmax(next_q_online[i])
                    targets[i, a] = r + gamma * next_q_target[i, best_action]
                else:
                    # Standard DQN
                    targets[i, a] = r + gamma * np.max(next_q_target[i])

        model.fit(states, targets, epochs=1, verbose=0)

      if done:
        break

    # Update target network occasionally
    if episode % 5 == 0:
      target_model.set_weights(model.get_weights())

    epsilon = max(epsilon_min, epsilon * epsilon_decay)
    rewards_history.append(total_reward)

    print(f"Episode {episode+1:2d} | Reward: {total_reward:3.0f} | Epsilon: {epsilon:.3f}")

  return rewards_history

# ============================================================
# TRAINING
# ============================================================

print("Training Standard DQN...")
dqn_rewards = train_agent(dqn_model())

print("Training DDQN...")
ddqn_rewards = train_agent(dqn_model(), ddqn=True)
```

```
print("Training Dueling DQN...")
dueling_rewards = train_agent(dueling_model())

print("Training Prioritized Replay...")
prioritized_rewards = train_agent(dqn_model(), prioritized=True)

# ============================================================
# RESULTS
# ============================================================

print("\n" + "=" * 60)
print("PERFORMANCE COMPARISON")
print("=" * 60)
print("DQN Average Reward      :", round(np.mean(dqn_rewards), 2))
print("DDQN Average Reward     :", round(np.mean(ddqn_rewards), 2))
print("Dueling DQN Average     :", round(np.mean(dueling_rewards), 2))
print("Prioritized Replay Avg  :", round(np.mean(prioritized_rewards), 2))

# ============================================================
# VISUALIZATION
# ============================================================

plt.figure(figsize=(10, 6))
plt.plot(dqn_rewards, label="DQN")
plt.plot(ddqn_rewards, label="DDQN")
plt.plot(dueling_rewards, label="Dueling DQN")
plt.plot(prioritized_rewards, label="Prioritized Replay")
plt.title("Comparison of DQN Variants")
plt.xlabel("Episode")
plt.ylabel("Reward")
plt.legend()
plt.grid(True)
plt.show()
```

```
# Average Reward Comparison
methods = ["DQN", "DDQN", "Dueling DQN", "Prioritized Replay"]
averages = [
    np.mean(dqn_rewards),
    np.mean(ddqn_rewards),
    np.mean(dueling_rewards),
    np.mean(prioritized_rewards)
]

plt.figure(figsize=(9, 5))
plt.bar(methods, averages)
plt.title("Average Reward Comparison")
plt.xlabel("DQN Variant")
plt.ylabel("Average Reward")
plt.xticks(rotation=15)
plt.grid(axis="y")
plt.show()

env.close()
```

OUTPUT:-

```
Monte Carlo State Values
--------------------------------
Start: 0.000
Middle: 10.000
Goal: 0.000

Learned Policy
--------------------------------
Start -> Move
Middle -> Move
Goal -> Stay
```



Visualization

The program produces two graphs:

1. Comparison of DQN Variants
   Shows the reward obtained by DQN, DDQN, Dueling DQN, and Prioritized Experience Replay over the training episodes.

2. Average Reward Comparison
   Compares the average reward achieved by each method.

Summary

Different DQN variants were implemented and compared. DDQN addresses Q-value overestimation, Dueling DQN separates state value and action advantage, and Prioritized Experience Replay gives greater importance to selected experiences during training.

Result

Thus, DQN, DDQN, Dueling DQN, and Prioritized Experience Replay were successfully implemented and compared, and their learning performance was evaluated using reward-based visualizations.